

Create New User Account & Create New MySQL Connection & Test Connection

Query 1 Administration - Users and Privileges x

 Rafay Waseem
Users and Privileges

User Accounts

User	From Host
mysql.infoschema	localhost
mysql.session	localhost
mysql.sys	localhost
newuser	%
root	localhost

Details for account newuser@%

Login Account Limits Administrative Roles Schema Privileges

Login Name: You may create multiple accounts with the same name to connect from different hosts.

Authentication Type: For the standard password and/or host based authentication, select 'Standard'.

Limit to Hosts Matching: % and _ wildcards may be used

Password: Type a password to reset it.

Confirm Password: Enter password again to confirm.

Medium strength password.

Expire Password

 Setup New Connection

Connection Name: Type a name for the connection

Connection Method: Method to use to connect to the RDBMS

Parameters SSL Advanced

Hostname: Port: Name or IP address of the server host - and TCP/IP port.

Username: Name of the user to connect with.

Password: The user's password. Will be requested later if it's not set.

Default Schema: The schema to use as default schema. Leave blank to select it later.

Configure Server Management... Test Connection Cancel OK

MySQL Connections (+)

Rafay Waseem

 root

 127.0.0.1:3306



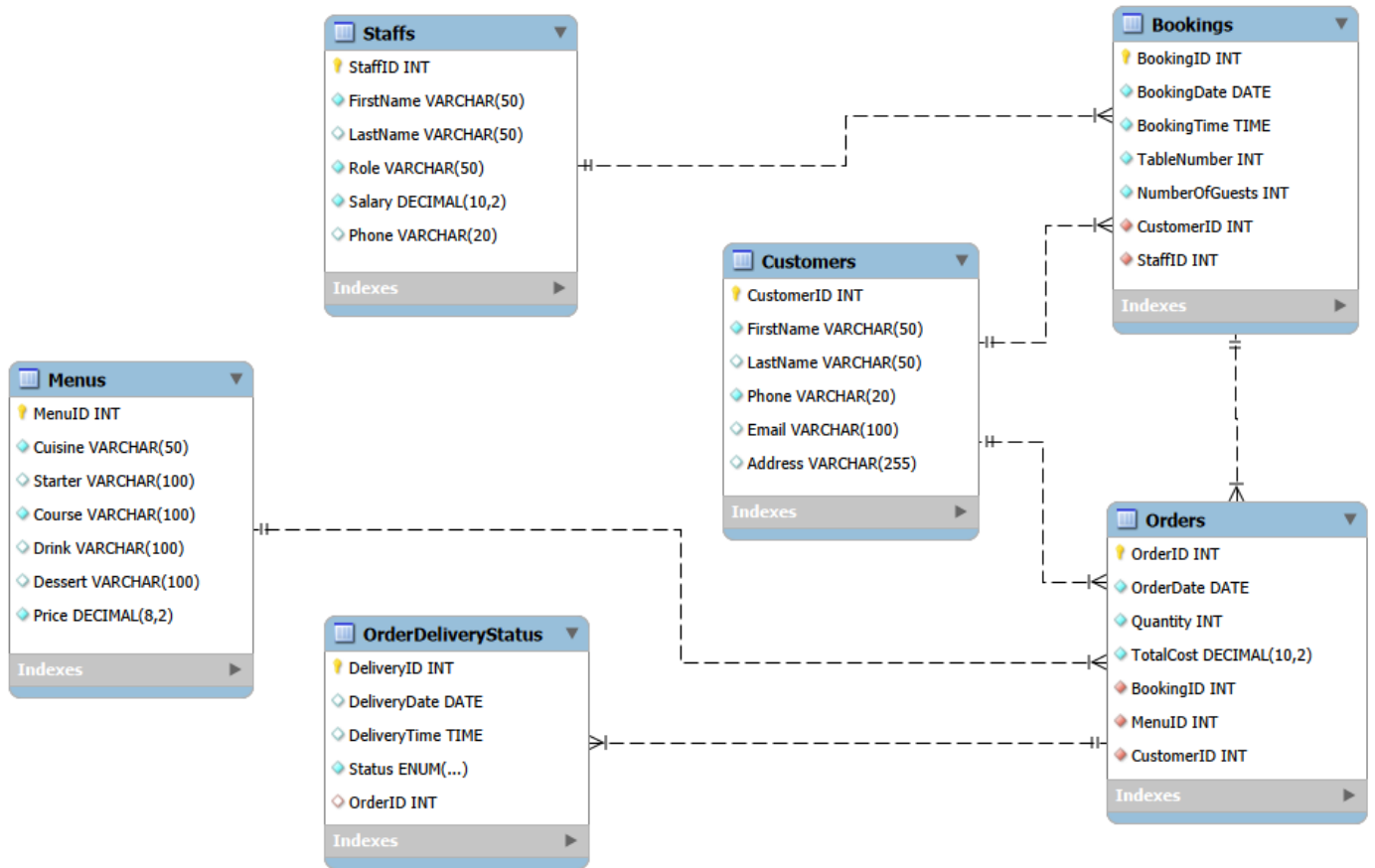
Rafay_Waseem

 Rafay_Waseem...

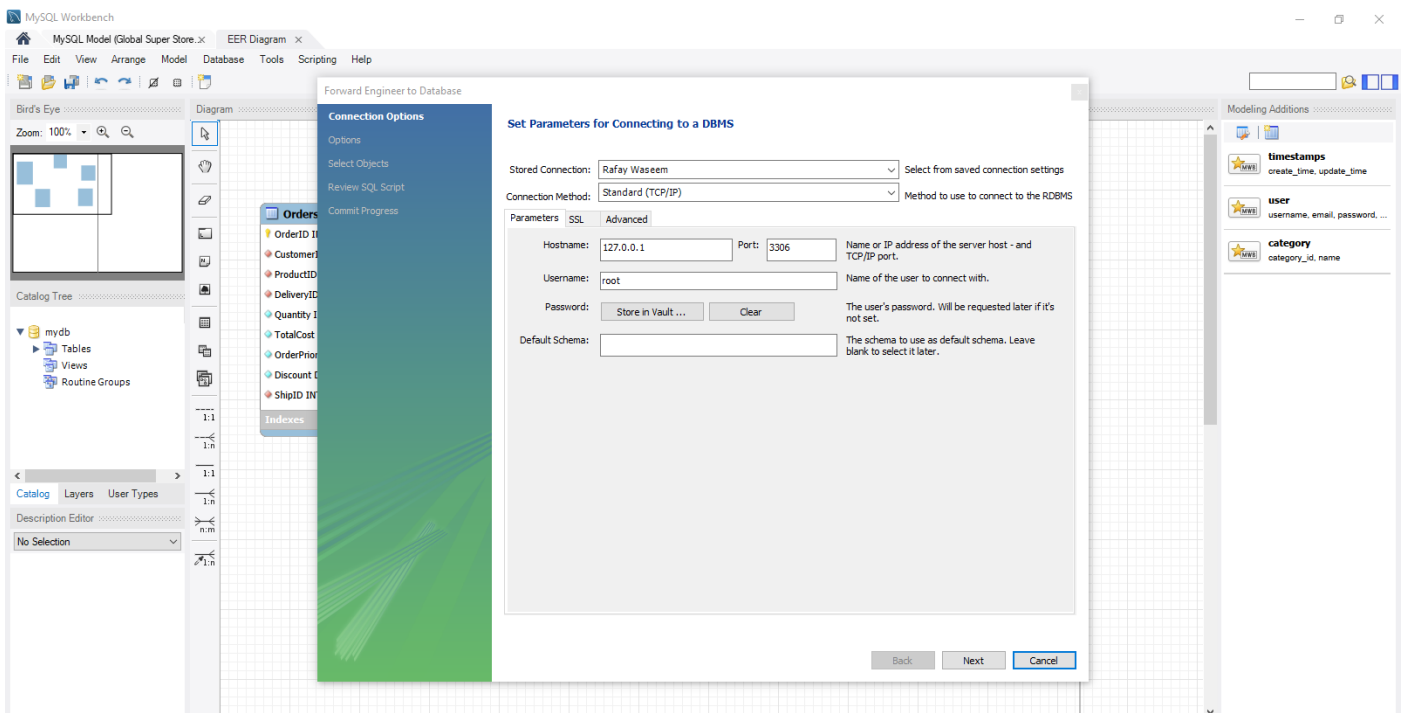
 127.0.0.1:3306

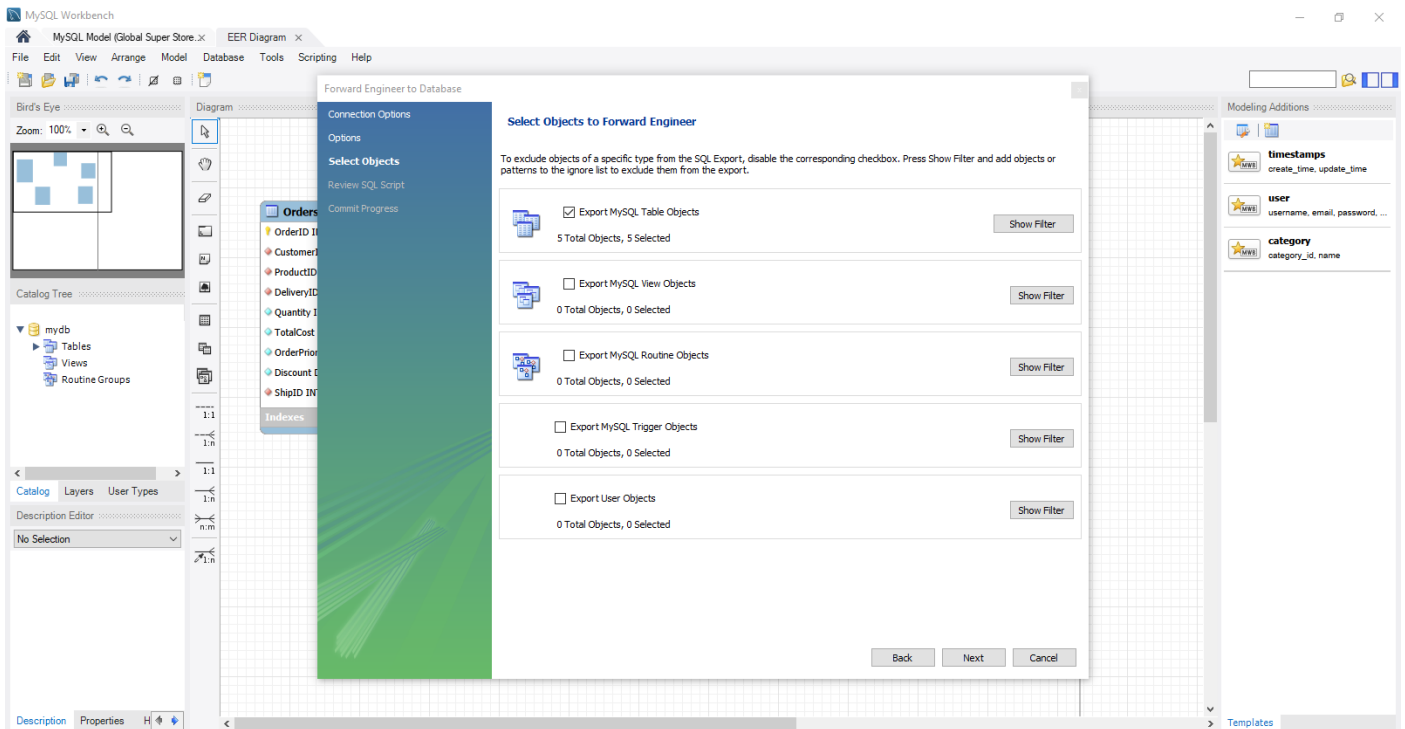


LittleLemonDM ER Diagram Schema



Implement the data model - Forward Engineer





Forward Engineer to Database

Connection Options
Options
Select Objects
Review SQL Script
Commit Progress

Review the SQL Script to be Executed

This script will now be executed on the DB server to create your databases.
You may make changes before executing.

```

1      -- MySQL Workbench Forward Engineering
2
3      SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
4      SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
5      SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES';
6
7      -----
8      -- Schema LittleLemonDM
9      -----
10
11     -----
12     -- Schema LittleLemonDM
13     -----
14     CREATE SCHEMA IF NOT EXISTS `LittleLemonDM` DEFAULT CHARACTER SET utf8 ;
15     USE `LittleLemonDM` ;
16
17     -----
18     -- Table `LittleLemonDM`.`Customers`
19     -----
20     CREATE TABLE IF NOT EXISTS `LittleLemonDM`.`Customers` (
21       `CustomerID` INT NOT NULL AUTO_INCREMENT,
22       `FirstName` VARCHAR(50) NOT NULL,
23       `LastName` VARCHAR(50) NULL,
24       `Phone` VARCHAR(20) NOT NULL,

```

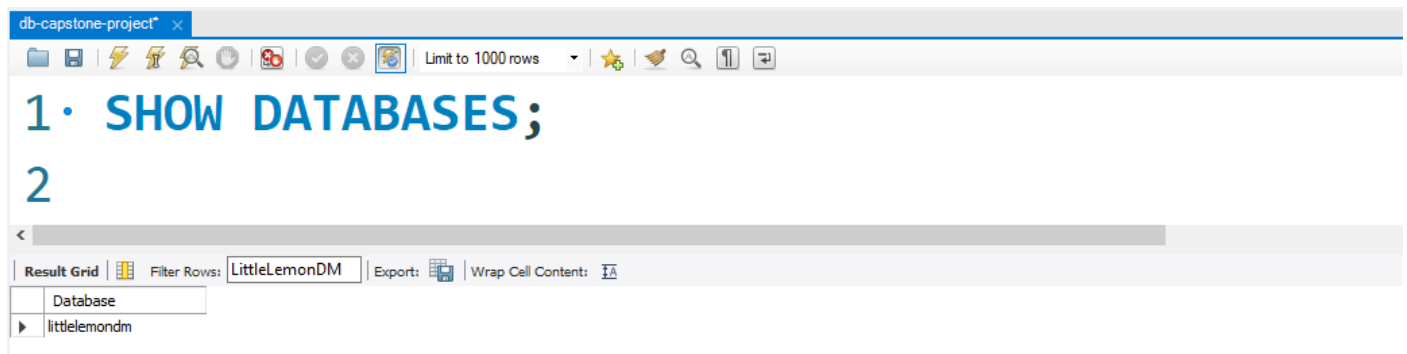
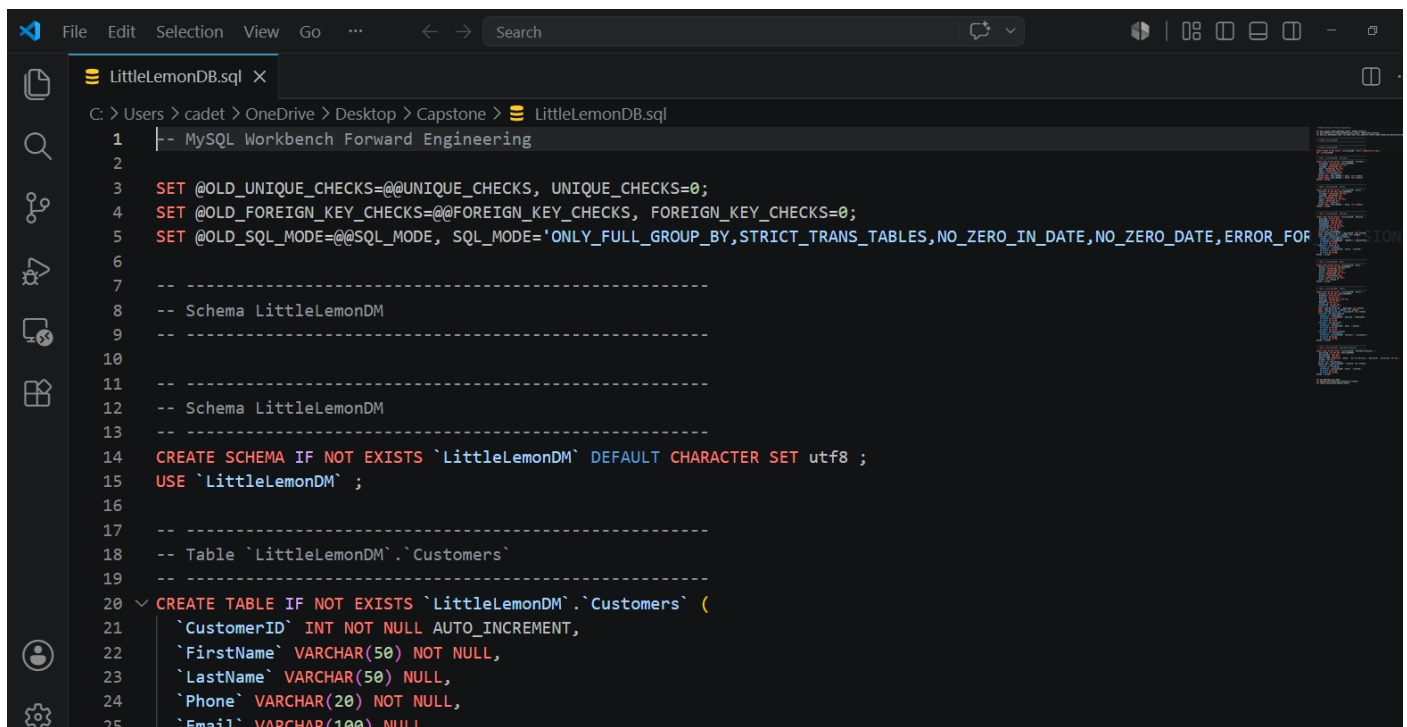
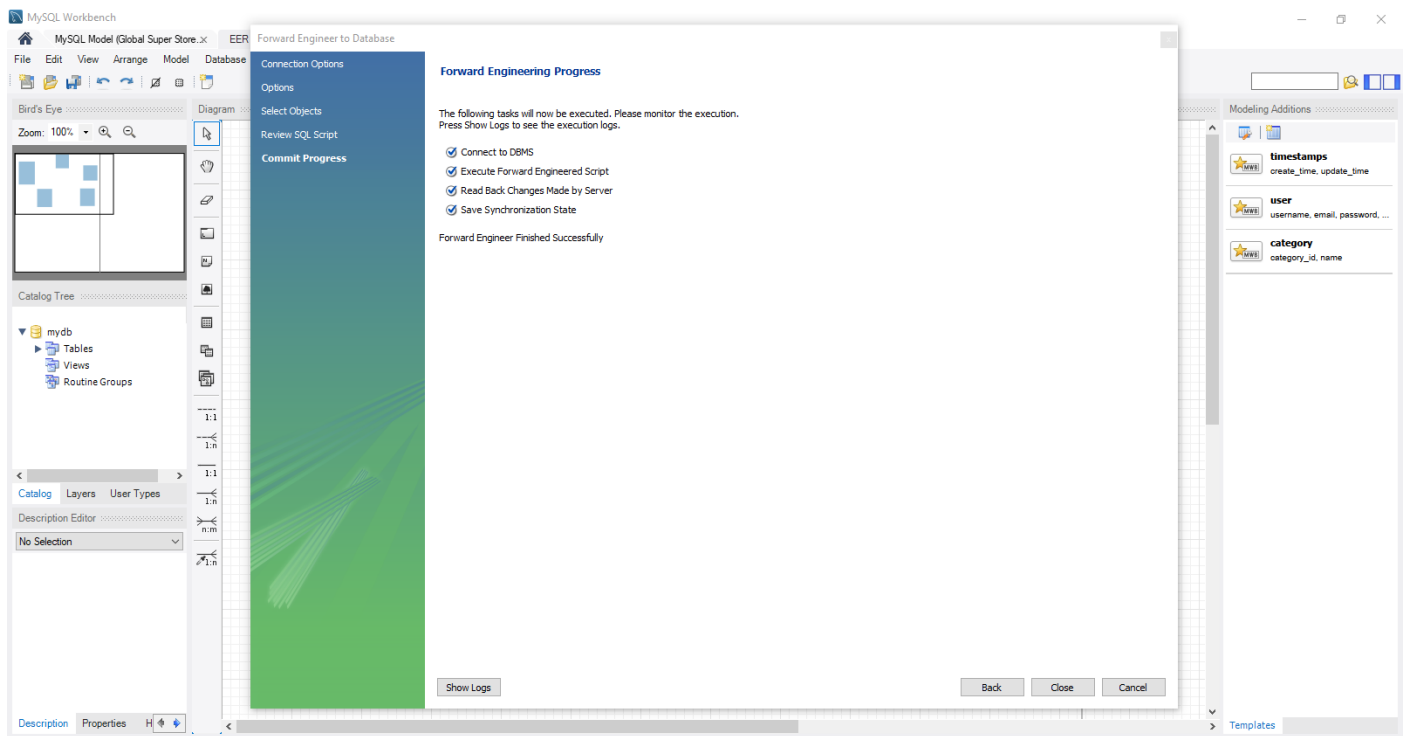
Save to File...

Copy to Clipboard

Back

Next

Cancel



db-capstone-project x

Limit to 1000 rows

2. USE LittleLemonDM;

3. SHOW TABLES;

Result Grid | Filter Rows: | Export: | Wrap Cell Content: A

Tables_in_littlelemondm
bookings
customers
menus
orderdeliverystatus
orders
staffs

Insert Records in the table

MySQL Workbench

Rafay Vaseem x

File Edit View Query Database Server Tools Scripting Help

Navigator: SQL File 5 Data Insertions

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

```
1
2
3 • USE LittleLemonDM;
4
5 -----
6 -- Insert into Customers
7 -----
8 • INSERT INTO Customers (FirstName, LastName, Phone, Email, Address) VALUES
9 ('John', 'Smith', '+1-202-555-0101', 'john.smith@email.com', '123 Maple St, Washington, DC 20001'),
10 ('Jane', 'Doe', '+1-202-555-0102', 'jane.doe@email.com', '456 Oak Ave, Washington, DC 20002'),
11 ('Michael', 'Johnson', '+1-202-555-0103', 'michael.j@email.com', '789 Pine Rd, Washington, DC 20003'),
12 ('Emily', 'Williams', '+1-202-555-0104', 'emily.w@email.com', '321 Elm St, Washington, DC 20004'),
13 ('David', 'Brown', '+1-202-555-0105', 'david.b@email.com', '654 Cedar Ln, Washington, DC 20005'),
14 ('Sarah', 'Jones', '+1-202-555-0106', 'sarah.j@email.com', '987 Birch Blvd, Washington, DC 20006'),
15 ('James', 'Garcia', '+1-202-555-0107', 'james.g@email.com', '147 Spruce Dr, Washington, DC 20007'),
16 ('Patricia', 'Miller', '+1-202-555-0108', 'patricia.m@email.com', '258 Willow Way, Washington, DC 20008'),
17 ('Robert', 'Davis', '+1-202-555-0109', 'robert.d@email.com', '369 Aspen Ct, Washington, DC 20009'),
18 ('Linda', 'Rodriguez', '+1-202-555-0110', 'linda.r@email.com', '741 Poplar Pl, Washington, DC 20010'),
19 ('William', 'Martinez', '+1-202-555-0111', 'william.m@email.com', '852 Magnolia St, Washington, DC 20011'),
20 ('Barbara', 'Hernandez', '+1-202-555-0112', 'barbara.h@email.com', '963 Cherry Ave, Washington, DC 20012'),
21 ('Richard', 'Lopez', '+1-202-555-0113', 'richard.l@email.com', '159 Peach Rd, Washington, DC 20013'),
22 ('Susan', 'Wilson', '+1-202-555-0114', 'susan.w@email.com', '357 Orange Ln, Washington, DC 20014'),
23 ('Joseph', 'Anderson', '+1-202-555-0115', 'joseph.a@email.com', '753 Banana Blvd, Washington, DC 20015'),
24 ('Karen', 'Thomas', '+1-202-555-0116', 'karen.t@email.com', '951 Grape Dr, Washington, DC 20016'),
25 ('Charles', 'Taylor', '+1-202-555-0117', 'charles.t@email.com', '246 Watermelon Way, DC 20017'),
26 ('Nancy', 'Moore', '+1-202-555-0118', 'nancy.m@email.com', '135 Kiwi Ct, Washington, DC 20018'),
```

Connect to the database using a Python client

```
db.py  ×
db.py > get_connection
1  import mysql.connector
2  from mysql.connector import Error
3
4  def get_connection():
5      try:
6          # Establish the connection
7          connection = mysql.connector.connect(
8              host="localhost",
9              user="root",
10             password="",
11             database="LittleLemonDM"
12         )
13
14         if connection.is_connected():
15             print("Successfully connected to the database")
16
17         # Create a cursor object
18         cursor = connection.cursor()
19
20         return connection, cursor
21
22     except Error as e:
23         print(f"Error while connecting to MySQL: {e}")
24
```

**Create the following stored procedures:
GetMaxQuantity(), ManageBooking(), UpdateBooking(),
AddBooking(), and CancelBooking().**

```
server.py > ...
1  from db import get_connection
2  from mysql.connector import Error
3
4  def create_procedures():
5
6      connection = get_connection()
7      if connection is None:
8          return
9
10     cursor = connection.cursor()
11
12     procedures = [
13         """
14         CREATE PROCEDURE GetMaxQuantity()
15         BEGIN
16             SELECT MAX(Quantity) AS MaxQuantity FROM Orders;
17         END;
18         """,
19
20         """
21         CREATE PROCEDURE ManageBooking(
22             IN p_BookingID INT
23         )
24
```

server.py > ...

```
4 def create_procedures():
24     BEGIN
25         SELECT * FROM Bookings WHERE BookingID = p_BookingID;
26     END;
27     "",
28
29     ""
30     CREATE PROCEDURE AddBooking(
31         IN p_BookingID INT,
32         IN p_BookingDate DATE,
33         IN p_BookingTime TIME,
34         IN p_TableNumber INT,
35         IN p_NumberOfGuests INT,
36         IN p_CustomerID INT,
37         IN p_StaffID INT
38     )
39     BEGIN
40         INSERT INTO Bookings (BookingID, BookingDate, BookingTime, TableNumber, NumberOfGuests, CustomerID, Staf
41         VALUES (p_BookingID, p_BookingDate, p_BookingTime, p_TableNumber, p_NumberOfGuests, p_CustomerID, p_Staf
42     END;
43     "",
44
```

server.py > ...

```
4 def create_procedures():
45     ""
46     CREATE PROCEDURE UpdateBooking(
47         IN p_BookingID INT,
48         IN p_BookingDate DATE,
49         IN p_BookingTime TIME,
50         IN p_TableNumber INT,
51         IN p_NumberOfGuests INT,
52         IN p_CustomerID INT,
53         IN p_StaffID INT
54     )
55     BEGIN
56         UPDATE Bookings
57         SET BookingDate = p_BookingDate,
58             BookingTime = p_BookingTime,
59             TableNumber = p_TableNumber,
60             NumberOfGuests = p_NumberOfGuests,
61             CustomerID = p_CustomerID,
62             StaffID = p_StaffID
63         WHERE BookingID = p_BookingID;
64     END;
65     "",
```

server.py > ...

```
4 def create_procedures():
67
68     CREATE PROCEDURE CancelBooking(
69         IN p_BookingID INT
70     )
71     BEGIN
72         DELETE FROM Bookings WHERE BookingID = p_BookingID;
73     END;
74     """
75 ]
76
77 for proc_sql in procedures:
78     try:
79         cursor.execute(proc_sql)
80         print(f"Successfully created procedure: {proc_sql.split()[2]}") # get procedure name
81     except Error as e:
82         # If the procedure already exists
83         print(f"Error creating procedure: {e}")
84
85 connection.commit()
86 cursor.close()
87 connection.close()
88
89 if __name__ == "__main__":
90     create_procedures()
```

Successfully connected to the database

Successfully created procedure: GetMaxQuantity()

Successfully created procedure: ManageBooking(

Successfully created procedure: AddBooking(

Successfully created procedure: UpdateBooking(

Successfully created procedure: CancelBooking(

(.venv) PS C:\Users\cadet\OneDrive\Desktop\Capstone\Python Code>

Call Procedure

```
1 • USE LittleLemonDM;
2
3 -- -----
4 -- 1. GetMaxQuantity - returns the highest quantity from Orders
5 -- -----
6 • CALL GetMaxQuantity();
7
8 -- -----
9 -- 2. ManageBooking - retrieve booking details for a given ID
10 -- (example: BookingID = 1)
11 -- -----
12 • CALL ManageBooking(1);
13
14 -- -----
15 -- 3. AddBooking - insert a new booking (choose an unused BookingID)
16 -- (Here we use 31, adjust if 31 already exists)
17 -- -----
18 • CALL AddBooking(31, '2026-06-16', '19:00:00', 10, 2, 1, 3);
19
20 -- Verify the insertion
21 • SELECT * FROM Bookings WHERE BookingID = 31;
```

```

Call Procedure x
Limit to 1000 rows
16 -- (Here we use 31, adjust if 31 already exists)
17 -----
18 • CALL AddBooking(31, '2026-06-16', '19:00:00', 10, 2, 1, 3);
19
20 -- Verify the insertion
21 • SELECT * FROM Bookings WHERE BookingID = 31;
22 |
23 -----
24 -- 4. UpdateBooking - update the booking we just added
25 -----
26 • CALL UpdateBooking(31, '2026-06-17', '20:30:00', 12, 4, 2, 4);
27
28 -- Check the update
29 • SELECT * FROM Bookings WHERE BookingID = 31;
30
31 -----
32 -- 5. CancelBooking - delete the booking (clean up)
33 -----
34 • CALL CancelBooking(31);
35
36 -- Confirm deletion
37 • SELECT * FROM Bookings WHERE BookingID = 31;

```

Results of Procedures

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
MaxQuantity			
3			

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	BookingID	BookingDate	BookingTime	TableNumber	NumberOfGuests	CustomerID	StaffID
	1	2026-06-01	18:30:00	5	2	1	3

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	BookingID	BookingDate	BookingTime	TableNumber	NumberOfGuests	CustomerID	StaffID
	31	2026-06-16	19:00:00	10	2	1	3
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	BookingID	BookingDate	BookingTime	TableNumber	NumberOfGuests	CustomerID	StaffID
▶	31	2026-06-17	20:30:00	12	4	2	4
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	BookingID	BookingDate	BookingTime	TableNumber	NumberOfGuests	CustomerID	StaffID
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Interactive dashboard for sales and profits

Tableau - Book1

File Data Server Window Help

Connections

EgSK5enRQCuiucbZmZelA...
Microsoft Excel

Sheets

☐ Use Data Interpreter
Data Interpreter might be able to clean your Microsoft Excel workbook.

Orders

New Union

New Table Extension

LittleLemonDB

Connection: ☒ Live ☐ Extract

Filters: 0 | Add

Orders

23 fields 21000 rows

100 rows

Name: Orders

Description: No description available.

Fields

Type	Field Name	Phys...	Rem...
#	Row Nun	Orders	Row N...
Abc	Order ID	Orders	Order ...
Abc	Order Da	Orders	Order ...
Abc	Delivery	Orders	Delive...
Abc	Custome	Orders	Custo...
Abc	Custome	Orders	Custo...
Abc	City	Orders	City
Abc	Country	Orders	Country

Abc Orders	Abc Orders	Abc Orders	Abc Orders	Abc Orders	Abc Calculation	Abc Calculation
Cuisine Name	Starter Name	Desert Name	Drink	Sides	First Name	Last Name
Turkish	Falafel	Affogato	Roma Red wine	Focaccia	Wilek	Ivanyushkin
Greek	Minestrone	Turkish yoghurt	Ankara White Wine	Meatballs	Uriel	Grooby
Italian	Hummus	Baklava	Turkish Coffee	Fries	Tony	Crabtree
Italian	Minestrone	Cheesecake	Italian Coffee	Bruschetta	Tiffani	Halstead
Italian	Falafel	Greek yoghurt	Athens White wine	Tapas	Tiertza	Doogan
Greek	Falafel	Turkish yoghurt	Ankara White Wine	Meatballs	Tasia	Faulty
Italian	Olives	Greek yoghurt	Athens White wine	Tapas	Tamarah	Filasov
Italian	Minestrone	Affogato	Roma Red wine	Focaccia	Sorcha	Beesley
Italian	Hummus	Affogato	Roma Red wine	Focaccia	Son	Hurworth
Italian	Flatbread	Affogato	Roma Red wine	Focaccia	Skipp	Lowater
Turkish	Olives	Ice cream	Corfu Red Wine	Potato salad	Sibley	Rodder

Go to Worksheet

Data Source: Sheet 1

Tableau - Book1

File Data Server Window Help

Connections

EgSK5enRQCuiucbZmZelA...
Microsoft Excel

Sheets

☐ Use Data Interpreter
Data Interpreter might be able to clean your Microsoft Excel workbook.

Orders

New Union

New Table Extension

LittleLemonDB

Connection: ☒ Live ☐ Extract

Filters: 0 | Add

Orders

24 fields 21000 rows

100 rows

Name: On

Description: SUM([Sales]) - SUM([Cost])

Fields

Type	Field Name	Phys...	Rem...
#	Row Nun	Orders	Row N...
Abc	Order ID	Orders	Order ...
Abc	Order Da	Orders	Order ...
Abc	Delivery	Orders	Delive...
Abc	Custome	Orders	Custo...
Abc	Custome	Orders	Custo...
Abc	City	Orders	City
Abc	Country	Orders	Country

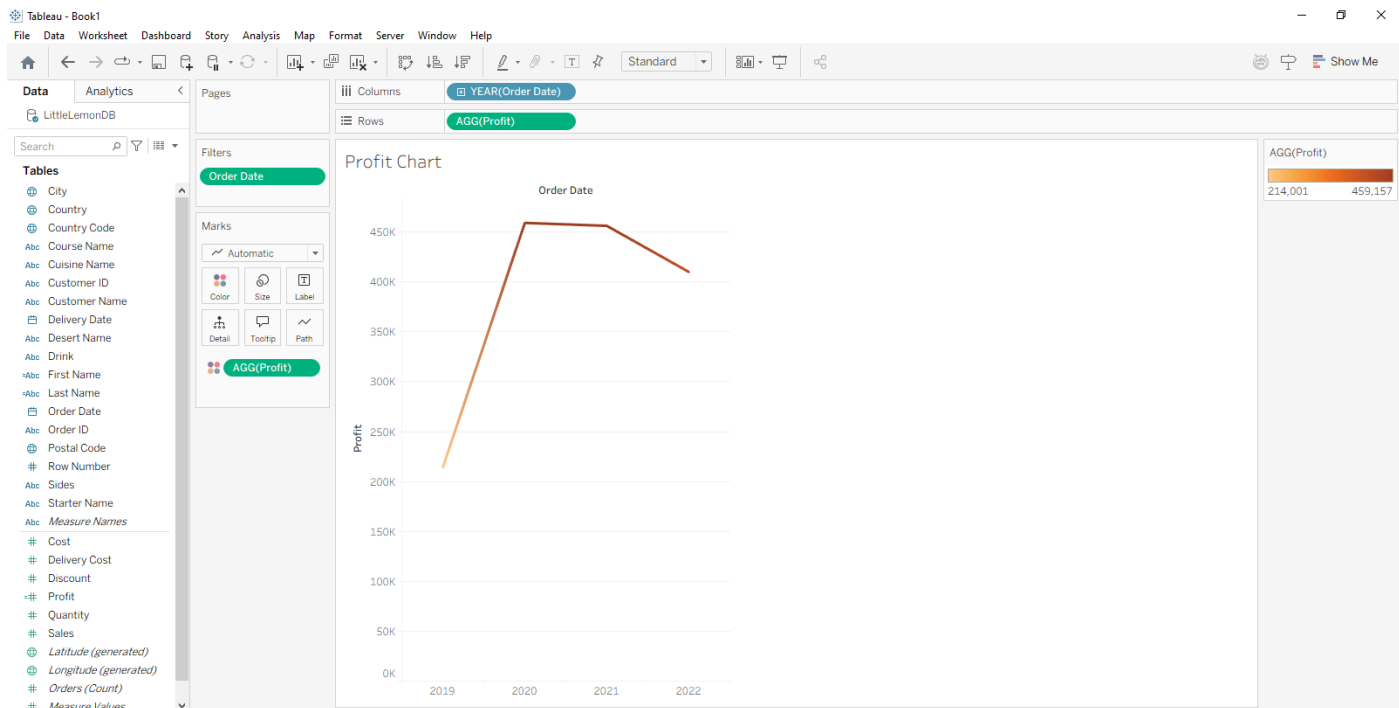
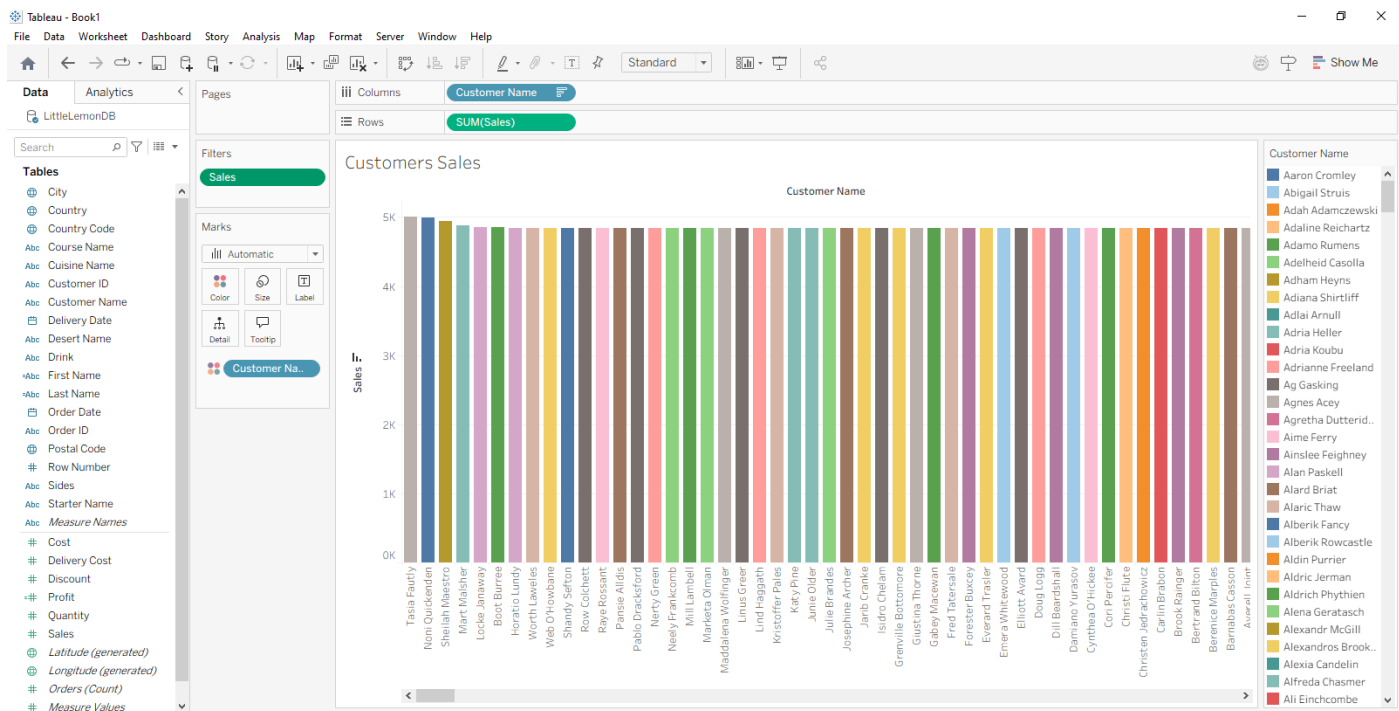
# Orders	# Orders	# Calculation	# Orders	# Orders	# Orders	Abc Orders	Abc Orders
Cost	Sales	Profit	Quantity	Discount	Delivery Cost	Course Name	Cu
320.000	480.000	160.000	2	51.0500	63.6400	Kabasa	Gri
320.000	480.000	160.000	2	31.2000	56.5100	Greek salad	Gri
320.000	480.000	160.000	2	30.0000	54.5900	Pizza	Gri
320.000	480.000	160.000	3	51.0500	31.5200	Greek salad	Ita
320.000	480.000	160.000	2	91.1400	29.5700	Kabasa	Gri
310.000	465.000	155.000	2	17.4000	56.9500	Greek salad	Ita
310.000	465.000	155.000	3	91.1400	95.6300	Pizza	Ita
310.000	465.000	155.000	2	24.0000	45.5700	Kabasa	Ita
310.000	465.000	155.000	2	17.4000	82.3400	Pizza	Gri
310.000	465.000	155.000	3	68.8900	30.6900	Greek salad	Ita
216282	352.500	117.500	1	15.0000	96.7500	Bean soup	Ita

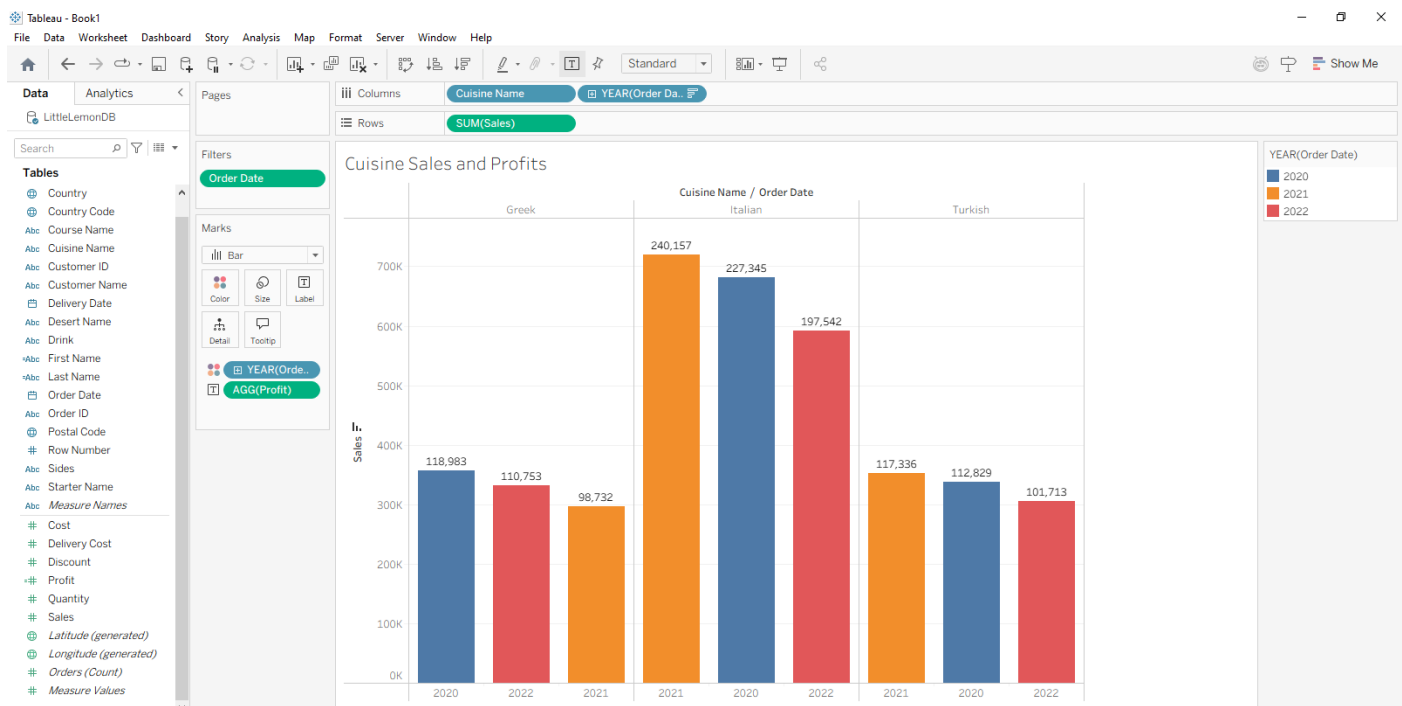
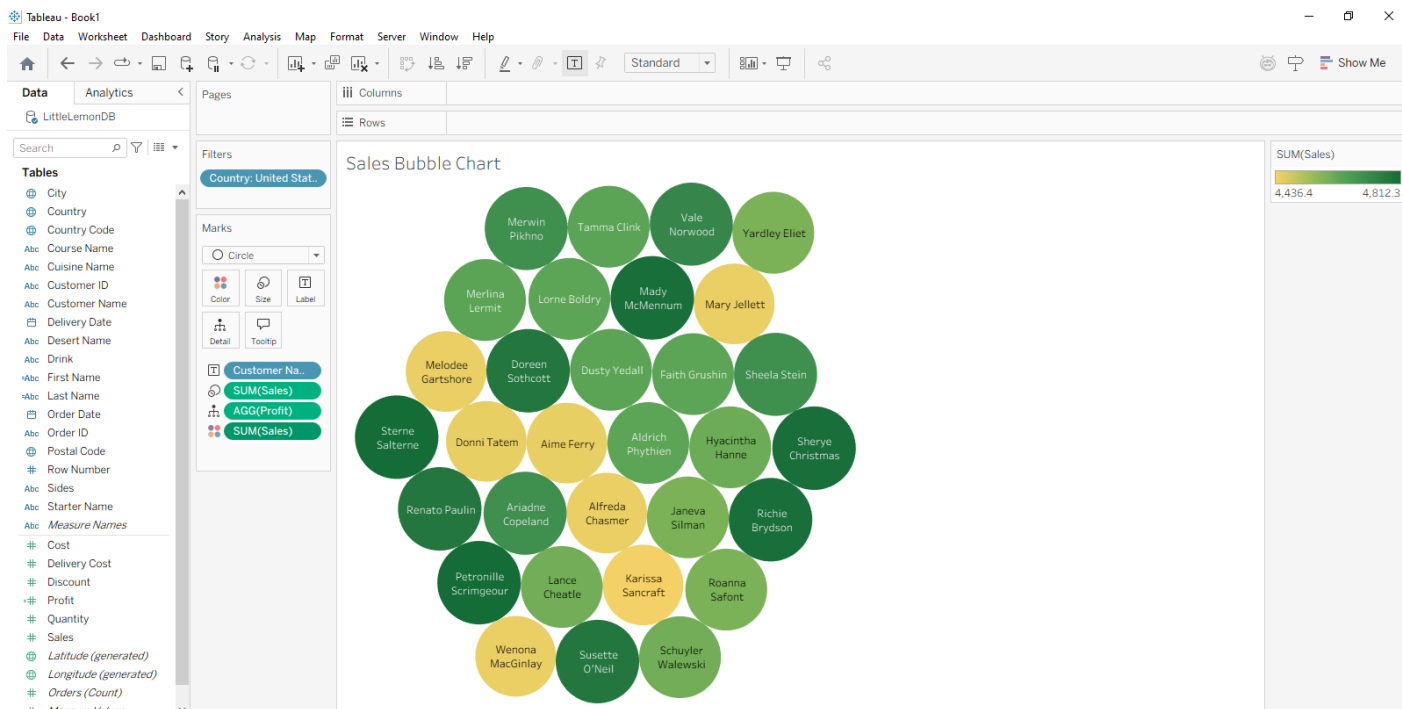
The calculation is valid.

Apply OK

Go to Worksheet

Data Source: Sheet 1





Dashboard

Layout

Default

Phone

Device Preview

Size

Desktop Browser (1000 x 800)

Sheets

- Customers Sales
- Profit Chart
- Sales Bubble ...
- Cuisine Sales a...

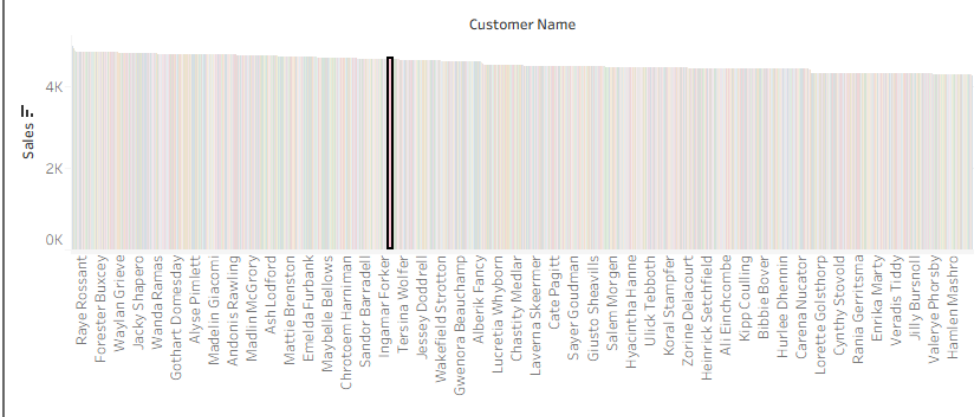
Objects

- Horizontal Container
- Vertical Container
- Text
- Extension
- Pulse Metric
- Image
- Blank
- Workflow
- Web Page

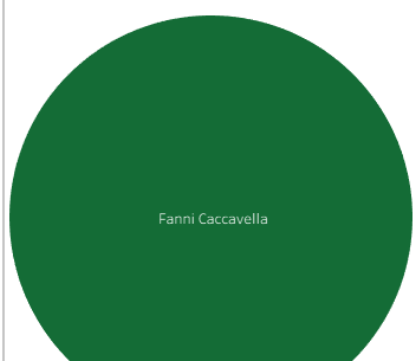
Tiled Floating

Show dashboard title

Customers Sales



Sales Bubble Chart



Customer Name
Aaron Cromley
Abigail Struis
Adah Adamczewski
Adaline Reichartz
Adamo Rumens
Adelheid Casolla
Adham Heyns
Adiana Shirtliff
Adlai Arnull
Adria Heller
Adria Koubu
Adrianne Freeland
Ag Gasking
Agnes Acey
Agretha Dutterid..
Aime Ferry
Ainslee Feighney
Alan Paskell
Alard Briat
Alaric Thaw
Alberik Fancy
Alberik Rowcastle
Aldin Purrier
Aldric Jerman
Aldrich Phythien
Alena Geratasch
Alexandr McGill
Alexandros Brook..
Alexia Candelin
Alfreda Chasmer
Ali Einchcombe
Alida Dumphy
Alisander Corneli..
Alisander Shoebo..
Alison Jamieson